StillCold — Final Presentation

Environmental monitoring without opening the door

Jeffrey Perdue · CSC 494 · Spring 2026

What is StillCold?

The problem: Every time you open a refrigerated enclosure to check the temperature, you change what you are measuring — and hide what is actually happening over time.

The solution: An HTU21D sensor wired to a Seeed XIAO ESP32-C6 exposes live temperature and humidity as BLE characteristics. A Flutter companion app polls those characteristics, persists readings locally, and provides a dashboard with alerts and trend charts.

Final system architecture

```
HTU21D ──I²C──▶ ESP32-C6 ──BLE──▶ Flutter App ──▶ User
```

Layer	Component
Sensor	HTU21D temperature + humidity (I^2C, 3.3 V)
MCU / BLE server	Seeed XIAO ESP32-C6
Transport	BLE GATT (custom service + characteristics)
App + storage	Flutter · Drift (SQLite) · fl_chart

Single-MCU design finalised in Sprint 2 — Arduino Nano and level shifter removed.

Live Demonstration

What you will see:

- StillCold advertising as **STILLCOLD** over BLE
- Flutter app discovering, connecting, and displaying live temperature and humidity
- Settings
- Trend charts

Demo Video

Link to Demo Video

Progress since S1P — Overview (Weeks 10–14)

Week	Theme	Outcome
10	Stabilise & Plan	Architecture docs, SRS gap analysis, Sprint 2 backlog
11	Cold-Environment Testing	3 scenarios validated; BLE stable through fridge door
12	Flutter Must + Should	7 features shipped + DB migrations; all core SRS done
13	Single-MCU Migration	Nano + level shifter removed; unified ESP32-C6 firmware
14	Trend Charts	1h/24h/7d charts, downsampling, retention; all SRS tiers complete

All Must + Should + Stretch SRS requirements complete.

What I Learned with AI — Topic 1

Hardware: A Sensor Output Is an Interpretation

The HTU21D does not hand you "the temperature." It:

1. Capacitively senses a physical property
2. Runs an internal ADC conversion
3. Applies a compensation algorithm defined in its datasheet
4. Delivers a formatted value over I^2C

Every step shapes what the number actually means. AI helped me work through the datasheet — conversion timing, the CRC error byte, the compensation curves — and connect classroom theory (ADC, sampling) to what I was observing on the bench.

> *The number is only as trustworthy as the chain that produced it.*

What I Learned with AI — Topic 1 (continued)

Cold Testing Made This Concrete

I wrapped the sensor in bubble wrap to protect it during the refrigerator test. The readings took far longer than expected to match the fridge air temperature.

Why: The bubble wrap insulated the sensor from the ambient air. The reading was accurate — for the sensor's immediate thermal environment. Not for the open fridge.

Design consequence: A reading stamped "2 seconds ago" can still be wrong for the current state of the environment. That insight directly produced the stale-data indicator — an amber warning when no fresh reading has arrived in more than two minutes.

> *Testing is the only way to know whether an interpretation is actually useful.*

What I Learned with AI — Topic 2

BLE GATT Is a Read Model, Not a Push Model

Early in the project I thought of BLE like a network subscription — you connect, you get events. AI helped me understand how GATT actually works:

- The **client** (app) reads characteristics on demand
- The **server** (MCU) holds values and waits
- Notify/Indicate *are* possible but require explicit setup — and still depend on a live connection

This drove every connection design decision:

Decision	Because
2-second polling timer	Client must pull; server never pushes
Stale indicator	If polling stops, the last reading silently ages
Reconnect-on-disconnect	MCU can lose power at any time; app must recover gracefully
Re-advertise in `onDisconnect`	MCU must be ready when the app comes back

What I Learned with AI — Topic 2 (continued)

Interface Stability Made a Hardware Refactor Invisible

From early in the project, AI helped me think about the BLE contract — service UUID, characteristic UUIDs, and ASCII value format — as a stable interface, independent of the hardware behind it.

Sprint 1: HTU21D → Nano → level shifter → ESP32-C6 → BLE

Sprint 2 Week 13: Entire hardware stack below the BLE layer was replaced.

```
Before: HTU21D → Nano (5 V) → level shifter → ESP32-C6 → BLE
After:  HTU21D → ESP32-C6 (3.3 V, I²C direct) → BLE
```

The Flutter app required zero changes.

Because the interface never changed, the app had no way to know the Nano ever existed. A complete hardware refactor was invisible to the software.

> *Good interface design is what makes components independently changeable.*

Issues Encountered — Hardware & Firmware

Issue 1: Wrong I²C pins in the migration plan

The architecture document listed the wrong GPIO numbers for I²C on the XIAO ESP32-C6. Before writing any firmware, I verified the actual pinout against the hardware diagram.

- The correct pins are **GPIO 22 (SDA / D4)** and **GPIO 23 (SCL / D5)**
- Because this was caught before wiring, the first compile was correct

There was also a subtle firmware trap: the SparkFun HTU21D library calls `Wire.begin()` internally. Using `Wire.begin(SDA, SCL)` in the sketch would have been silently overwritten. The correct call is `Wire.setPins()` *before* the library initializes — it stores the pin values without starting the bus.

Issue 2: Sensor error codes 998 / 999 not caught by the guard

The Sprint 1 firmware used `isnan()` to detect bad sensor reads. The SparkFun library does not return NaN on error — it returns the numeric values `998` (I^2C timeout) and `999` (bad CRC) as ordinary floats. The old guard let these through to BLE as `"998.00"`.

AI helped me find this by reading the library source. Fix: `temperature >= 998 || isnan(temperature)`.

Issues Encountered — Testing Revealed a UX Gap

Cold-environment test results (Week 11):

Scenario	Result
Steady-state (~1 h)	37.7–38.2 °F; BLE stable throughout
Door open 35 s	+0.9 °F; recovered in ~12 min
Door open 3 min	+2.6 °F; recovered in ~23 min

The problem testing revealed:

After a door event, the sensor continued reporting readings that were timestamped as "just now" — but those readings reflected a disrupted environment, not steady-state cold. The app had no way to signal this to the user.

The same problem applies to disconnects: if the device goes offline for ten minutes and then reconnects, the first fresh reading looks fine, but the user has no visibility into the gap.

Solution: Stale-data indicator — an amber "Stale · last updated X min ago" banner that appears whenever no reading has arrived in more than two minutes. This feature was not in the original spec. It came entirely from running the test.

Links

Resource	URL
Canvas	https://nku.instructure.com/courses/88152/pages/individual-project-jeffrey-perdue
GitHub	https://github.com/jeffreyperdue/stillcold-csc-494-iot-project
Learning with AI	https://github.com/jeffreyperdue/csc-494-learning-with-ai
Demo Video	https://www.youtube.com/shorts/DTpED5YB1A8

All weekly presentations, firmware, Flutter source, architecture docs, and cold-test reports are in the GitHub repository.

NKU Celebration of Student Research and Creativity

Thursday, April 23, 2026

StillCold will be presented as a live demonstration at the **NKU Celebration of Student Research and Creativity**.

StillCold Research Poster

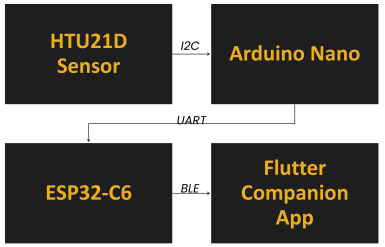
StillCold: Environmental Monitoring Without Opening the Door

Jeffrey Perdue, Dr. Samuel Cho
School of Computing and Analytics, College of Informatics, Northern Kentucky University

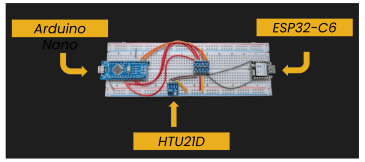
Introduction / Motivation

- Opening a refrigerator to check the temperature disrupts the environment and speeds spoilage; StillCold enables monitoring without opening the door.
- Bacteria can grow at slightly unsafe temperatures before food looks spoiled; StillCold provides continuous, early detection.
- Uses Bluetooth Low Energy (BLE) only—no Wi-Fi or cloud—keeping data local and running on a USB power bank even during outages.

System Architecture



Hardware

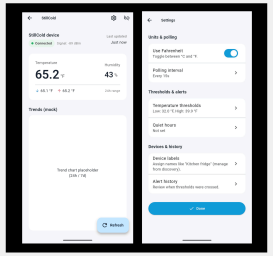


Component	Role
HTU21D Sensor	Measures temp & humidity inside the closed environment
Arduino Nano	Reads the sensor and passes the data along
ESP32-C6	Wirelessly broadcasts the latest temp & humidity to a smartphone via Bluetooth
Anker Power Bank	Powers the entire device

Flutter Companion App

- Runs on Android; discovers and connects to StillCold automatically over Bluetooth Low Energy
- Live dashboard displays current temperature and humidity — no manual refresh needed
- Readings timestamped and stored locally in SQLite (database) — no internet or cloud account required
- Customizable high/low temperature thresholds trigger alerts when conditions drift outside a safe range
- Full alert history log records when thresholds were crossed and the reading at that time

Flutter is a framework made by Google that allows you to write code once, and deploy it across multiple platforms (Android, iOS, Web)



Flutter companion app — live dashboard (left) and alert configuration (right)

How I Used AI to Solve the Problem

StillCold required learning several unfamiliar technologies simultaneously. AI served as an on-demand tutor and thought partner throughout the build.

Learning the Basics

- To learn C++ (the language powering the microcontrollers) and understand how a breadboard works — the platform used to wire and test the hardware prototype.

Building the Hardware

- To reason through wiring decisions, diagnose voltage mismatches, and evaluate component choices before testing them on the bench.

Building the App

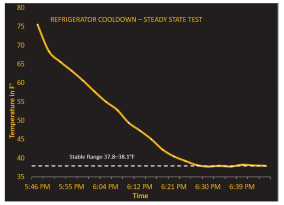
- To accelerate Flutter development, structure the codebase, and translate project requirements into working software features.

The Bigger Picture

- AI was most valuable for asking "why" before "how" — understanding tradeoffs rather than just getting answers. Every decision was ultimately verified through building and testing.

Cold Environment Testing

- Hardware sealed in a Ziploc bag with bubble wrap and placed in a household refrigerator.
- Three scenarios tested: steady-state cooldown, door open/close events, and post-power-cycle reconnection. **All three passed.**



Temperature recorded every 1–2 min via the Flutter app. Dashed line indicates stable fridge range (37.8–38.1°F).

Scenario	Key Result
Short door open (~35 sec)	+0.9°F peak, recovered in ~12 min
Prolonged Door Open	+2.6°F peak, recovered in ~23 min
Power Cycle (10 trials)	Valid reading in 10.8s avg (target: <15s)

Readings remained within ±0.25°F over 16 minutes at steady state — sufficient precision for food safety monitoring.

Next Steps

- Migrate hardware from breadboard to perfboard for a more durable assembly
- Explore single-MCU design by connecting the sensor directly to the ESP32-C6, removing the Arduino Nano
- Temperature history trend charts in the Flutter app
- iOS compatibility verification

Acknowledgements

The author thanks Dr. Samuel Cho for guidance and mentorship throughout this project. This work was supported by the School of Computing and Analytics, College of Informatics, Northern Kentucky University, and CINSAM.

Thank you

StillCold — *Environmental monitoring without opening the door*

- **Canvas:** https://nku.instructure.com/courses/88152/pages/individual-project-jeffrey-perdue
- **GitHub:** https://github.com/jeffreyperdue/stillcold-csc-494-iot-project

Questions?